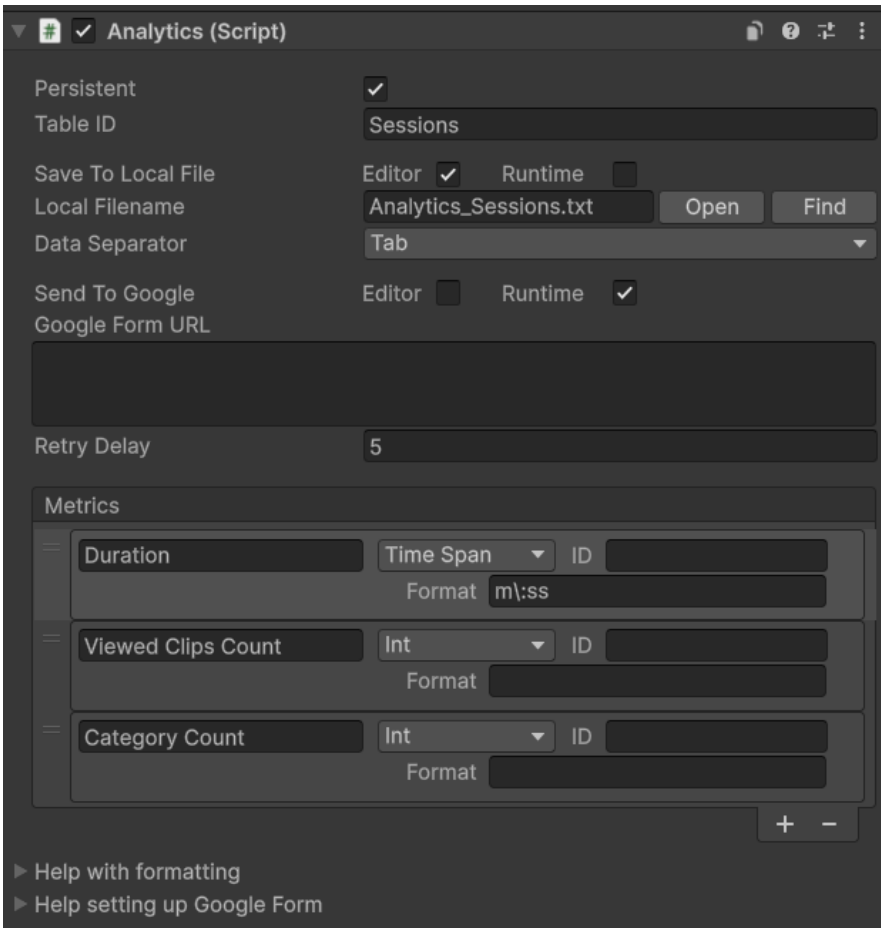
Barliesque.Analytics

This package leverages Google Forms and Spreadsheets to offer a simple system to collect and collate analytics data from Unity. For testing and offline use, data can be written to a local file in a variety of value-separated formats.

An Example Usage

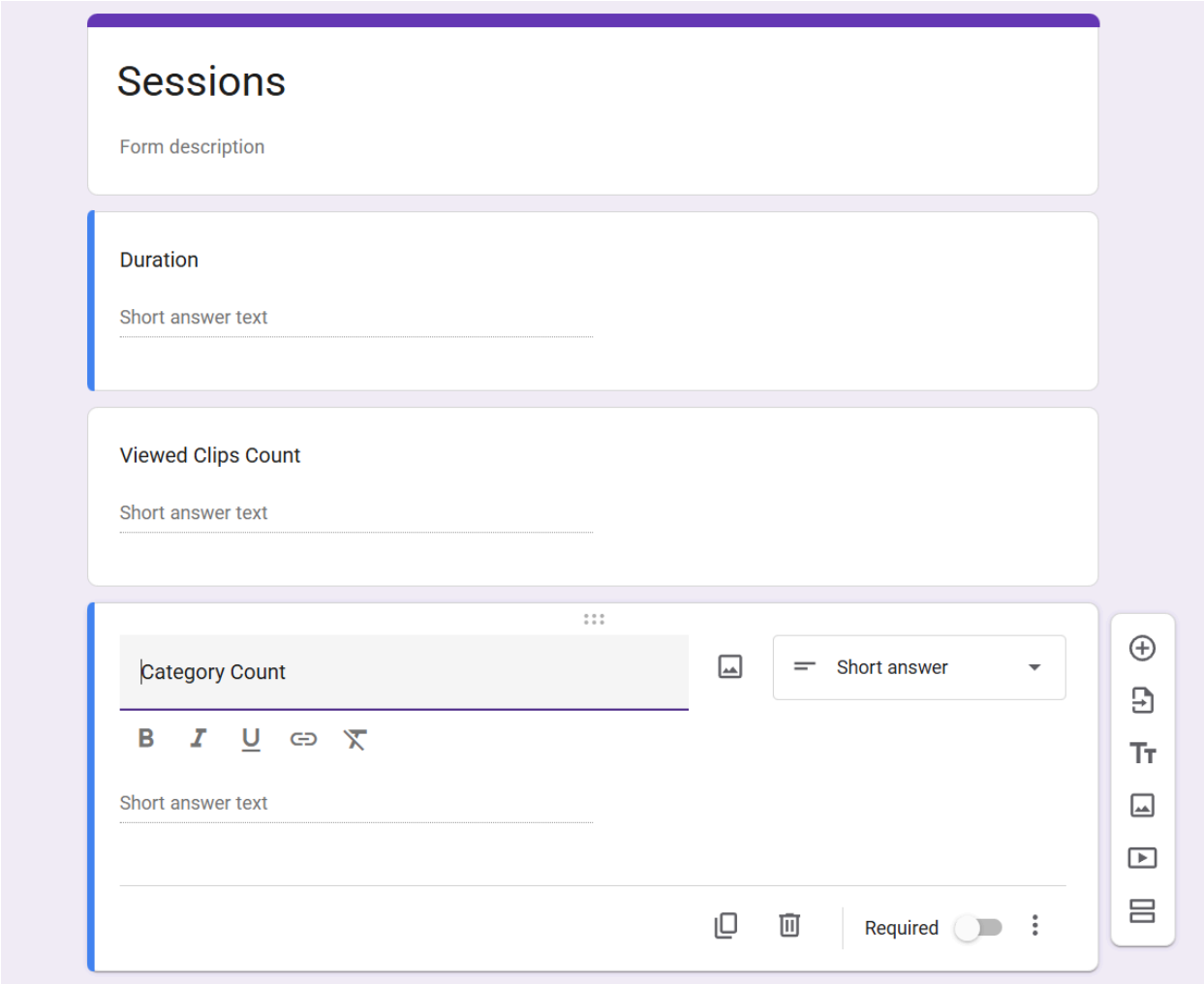
We'll look at the analytics implementation of a project that gives a short presentation to the user, followed by the opportunity to watch clips in a variety of categories. Our Analytics data will have two tables of data: Sessions and Clips. Sessions will have a single entry for each user's complete session, while Clips will have multiple entries for each session, taking note of every clip the user played. We'll create an Analytics component for each table. For Sessions, we want to know how long each user spent in the app, as well as a count of how many clips they watched, across how many categories.



In the example above, we've given the Table ID a name, set a local filename to record data while testing in the editor, and created an entry in the Metrics list for each of the fields we want to track. The Persistent option determines if this GameObject should be set to DontDestroyOnLoad. If you have a loader scene that's never unloaded, create your Analytics components there and then you can untick **Persistent**.

Setting up in Google

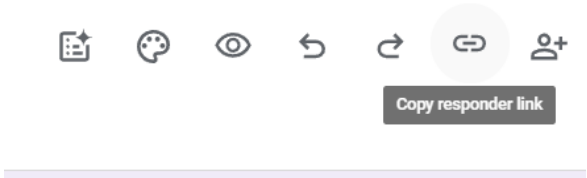
Once we get our Google Forms set up, we can finish filling in the missing data in the Analytics component. In Google Drive, we create a new Google Form, which looks like this:



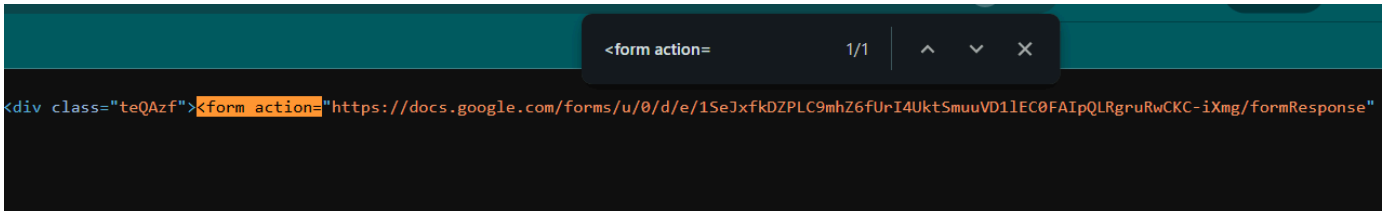
Regardless of type, all data will be sent to the Google Form as a string, so we must use the type **Short Answer** for all of the form questions. Note: When Google passes data from the form into

a spreadsheet, in addition to the fields from your form, a timestamp field is automatically added—So there's no need for us to define that metric ourselves.

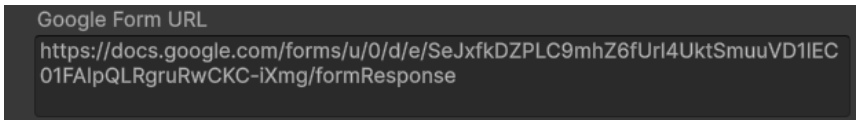
When you've created items for each of the fields, click the link icon at the top of the screen, and **Copy responder link:**



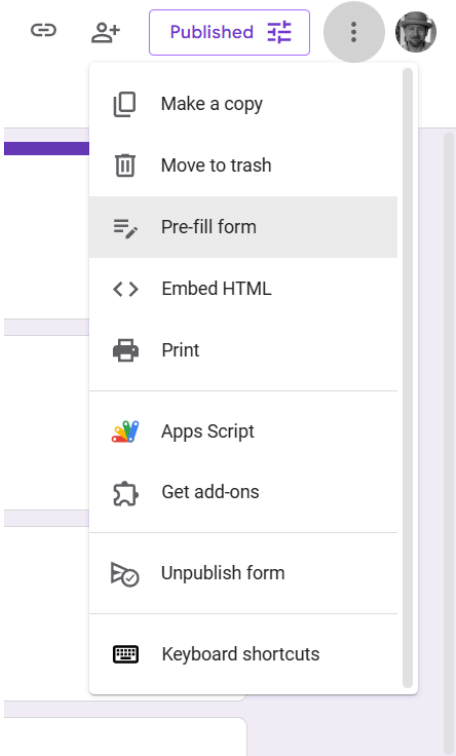
Next, open a new browser tab with the copied URL, and view the page source html. Search for "**<form action=**" to locate the submission URL.



Copy and paste that into the Google Form URL field of the Analytics component in Unity:



In the Google Form, click the three-dots menu button in the upper right corner, and select **Pre-fill form.**

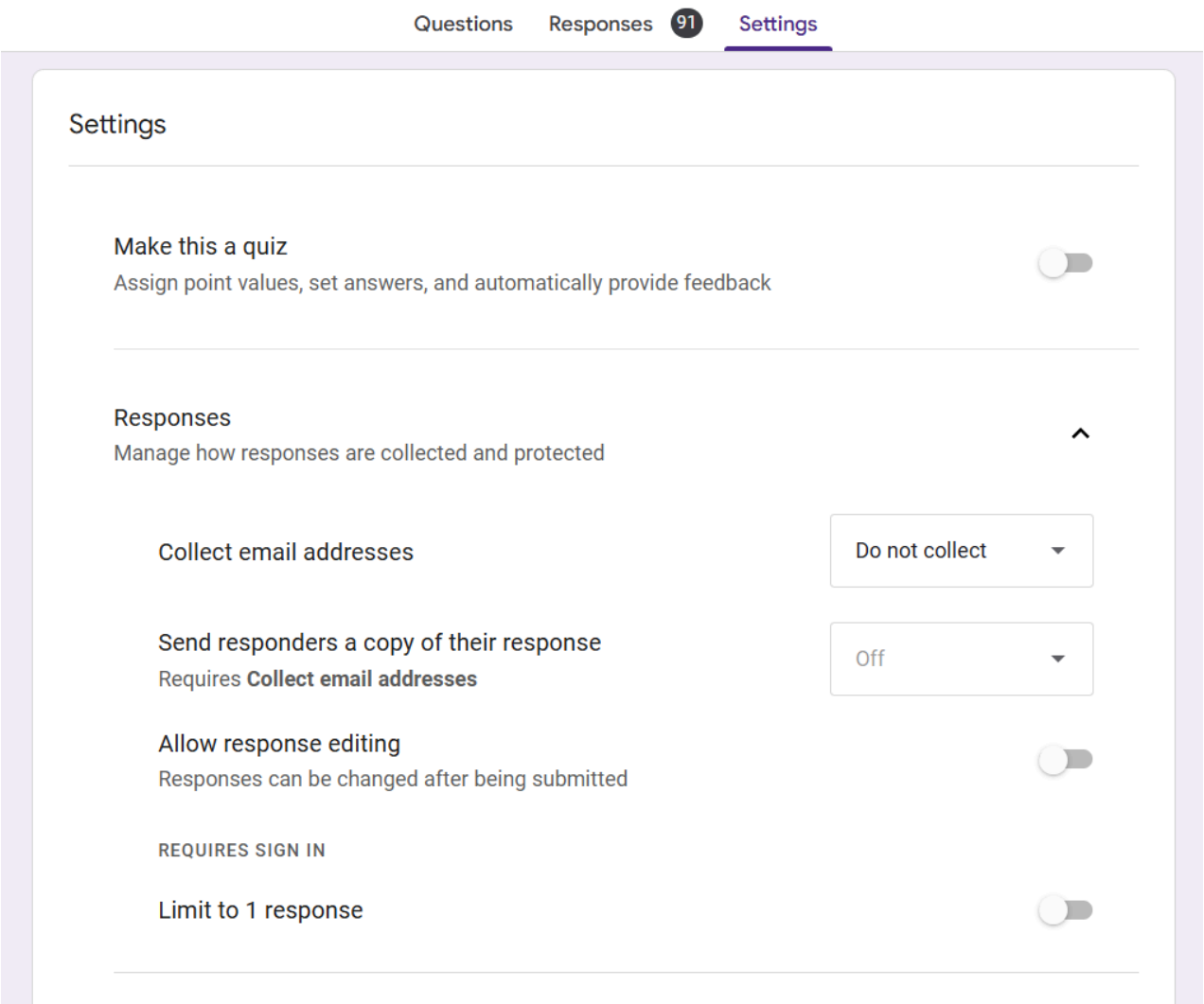


This opens up a special copy of the form. Right-click on each answer field and **Inspect**. Find the name property of an **<input>** element, e.g.: **name="entry.1797255114"**

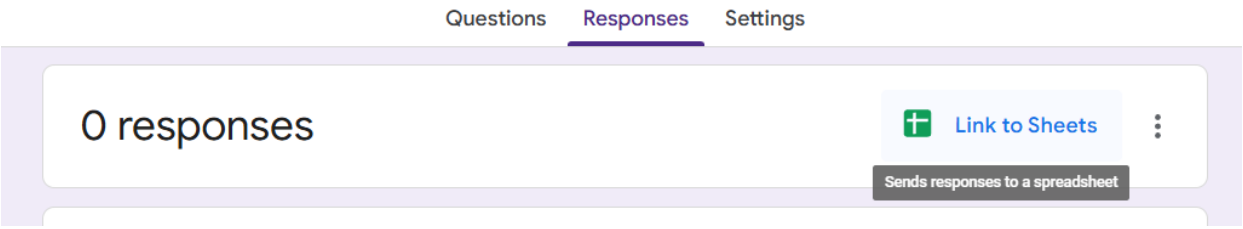
```
<div class="Xb9hP">  flex
  <input type="text" class="whsOnd zHQkBf" jsname="YPqjbf" autocomplete="off" tabindex="0" aria-
  label="Duration" aria-describedby="i.desc.1773372784 i.err.1773372784" name="entry.1797255114"
  value aria-disabled="false" dir="auto" data-initial-dir="auto" data-initial-value aria-invalid="fa
  lse"> == $0
  <div jsname="LwH6nd" class="ndJi5d snByac">Your answer</div>
</div>
```

Copy and paste the numeric part into the corresponding ID field in the Metrics list of the Analytics component.

When you've got all the field IDs copied, return to the Google Form and select the **Settings** tab. In the form settings, under **Responses** deselect all options requiring sign in.

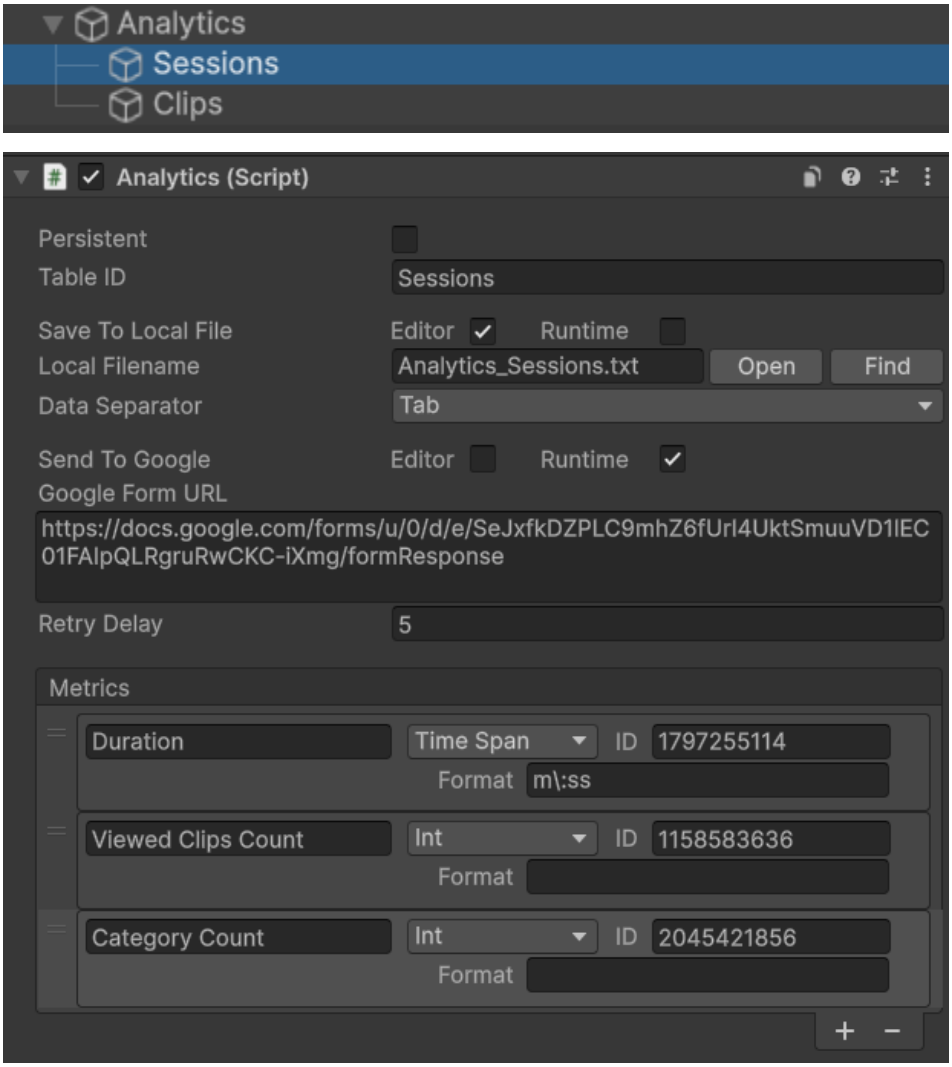


Lastly, we want to make sure all data recorded by the form is delivered to a spreadsheet, where it can be collated, digested and exported. To link the form results to a spreadsheet, click on the Responses tab and **Link to Sheets.**



The Analytics Component

We need one Analytics component for each table we created in Google. We'll put each Analytics component on its own GameObject, with the same name as the Table.



In projects with a single scene that gets reloaded, enable the **Persistent** option. When we send data to Analytics, we'll use the **Table ID** so it's applied to the correct table. The component can send data to Google Forms and/or save the data to a local file. By default, it will save to a data file while running in the Unity editor, and it will upload to Google running in a build. For initial testing, you may wish to enable the **Editor** option for **Send to Google** so you can verify data is being written correctly.

The **Retry Delay** parameter sets how long (in seconds) to wait before re-attempting to upload data to Google, in the event an error occurs, for example, due to a bad internet connection.

Each of the metrics we record can be formatted however you prefer. Leaving the Format field empty uses a default format which works well in most cases. Here are some examples of custom formats you can use.

Value Type	Format Code	Example Results	Notes
Fixed-point	`F2`	123.45 67.00 -4.32	
Percentage	`0%`	7% 32% 100%	
	`0.0%`	6.9% 32.1% 100.0%	`F1%` would yield the same result.
	`0.##%`	6.89% 32.1% 100%	
Currency	`$0.00`	$5.99 $500.10 $3600.00	
	`C`	$5.99 £500.10 ¥3600	Automatically applies local currency
Time	`h\:mm\:ss`	3:09:26	Minutes and seconds with leading zeros.
	`m\:ss.f`	9:26.3	`f` is the fractional part of seconds, rounded to one decimal place.
	`S.fff`	26.317	`fff` is the fractional part of seconds, rounded to three decimal places.

Note: There is a handy reference to this information, if you unfold **Help with Formatting** at the bottom of the component.

Implementing Analytics

We'll put all calls to Analytics in one class with a concise static API. This class will handle all the specifics of our implementation: keeping track of how many clips were viewed, and how many different categories the user selected. This way, implementing Analytics throughout the main project codebase will be greatly simplified.

```
using System.Collections.Generic;
using Barliesque.Analytics;
using UnityEngine;

public class PlayerAnalytics : MonoBehaviour
{

   private const string _sessionsTable = "Sessions";
   private const string _clipsTable = "Clips";

   static private List<string> _selectedCategories = new List<string>();

   static public void SessionStarted()
   {
      // Calling GameStarted resets all metrics to their default values and notes the time.
      Analytics.GameStarted(_sessionsTable, null);
      _selectedCategories.Clear();
   }

   static public void ClipPlayed(int clipID, string category)
   {
      Analytics.Metric(_clipsTable, "Clip ID").IntValue = clipID;
      Analytics.Metric(_clipsTable, "Category").StringValue = category;
      Analytics.RecordEntry(_clipsTable);

      // Keep track of unique categories and how many clips the user plays
      if (!_selectedCategories.Contains(category)) _selectedCategories.Add(category);
      Analytics.Metric(_sessionsTable, "Viewed Clips Count").IntValue++;
   }

   static public void SessionEnded()
   {
      Analytics.Metric(_sessionsTable, "Category Count").IntValue = _selectedCategories.Count;

      // Calling GameEnded writes out the session data.  Since it took note of when the session
      // started, it can now use that to determine the duration of the session--providing it
      // with a field here, sets the duration metric and then records the session entry.
      Analytics.GameEnded(_sessionsTable,"Duration");
   }

}
```

`Analytics.Metric()` returns a reference to an `AnalyticsMetric` object, which holds the current value of a given metric we defined in the Metrics list of the Analytics component. We can modify the value of a metric as many times as needed—its value is only used when calling either **`Analytics.GameEnded()`** or **`Analytics.RecordEntry()`**. `GameEnded()` is called once per session, while `RecordEntry()` writes the current metrics to a new record, typically multiple times during a session.